A Hybrid AsyncIO and Multiprocessing Approach for Bloom Filter Acceleration in Python

Alessio Poggesi
alessio.poggesi@edu.unifi.it

Abstract

This work presents a comparison between a sequential Bloom Filter implementation and a hybrid parallel version in Python that combines asyncio for asynchronous file I/O and multiprocessing for CPU-bound computation. The workload consists of constructing the Bloom Filter from large-scale datasets and performing membership queries, while varying input size, dataset fragmentation, number of worker processes, and chunking strategies.

The parallel design separates I/O-bound and CPU-bound tasks: file reading is handled asynchronously to avoid blocking the event loop, while hash computation and bit array updates are parallelized across multiple processes using a map–reduce approach. Experimental results show that, for sufficiently large datasets, the hybrid implementation achieves a stable speedup of approximately 4× using 8 worker processes on datasets of 10 and 20 million elements, demonstrating effective strong scaling and controlled parallel overhead. These findings highlight how a carefully balanced hybrid approach can overcome Python's Global Interpreter Lock limitations in data-intensive applications.

Future Distribution Permission

The author of this report gives permission for this document to be distributed to Unifi-affiliated students taking future courses.

1. Introduction

Bloom Filters are probabilistic data structures used to efficiently test set membership while maintaining a compact memory footprint. They are widely adopted in large-scale systems such as databases, caching mechanisms, and distributed applications, where fast membership queries are required and a limited probability of false positives is acceptable. A Bloom Filter relies on a fixed-size bit array and multiple hash functions to insert and query elements.

When applied to large datasets, Bloom Filters can become computationally demanding. Both insertion and query operations require the repeated evaluation of several hash functions per element, resulting in a predominantly CPU-bound workload. Additionally, reading large amounts of data from disk introduces a non-negligible I/O overhead.

In Python, these challenges are exacerbated by the Global Interpreter Lock (GIL), which limits parallel execution of CPU-bound threads. As a result, sequential implementations scale poorly to millions of elements, while naive parallelization may incur significant overhead.

This work investigates whether a hybrid parallel approach based on asyncio and multiprocessing can improve Bloom Filter performance compared to a sequential baseline by separating I/O-bound and CPU-bound tasks and identifying the configurations in which parallelization provides effective performance gains.

2. Algorithm Description

2.1. Sequential Bloom Filter Algorithm

A Bloom Filter is a probabilistic data structure that represents a set using a fixed-size bit array of length m and a set of k independent hash functions. Initially, all bits in the array are set to zero. Each hash function maps an input element to an index in the range $[0, m-1]$.

During the *insertion phase*, each element from the input dataset is processed sequentially. For a given element, the k hash functions are evaluated, producing k indices. The bits at these positions in

the bit array are then set to one. This operation does not require checking previous values, making insertion simple and efficient.

During the *query phase*, the same k hash functions are applied to the queried element. If all corresponding bits in the bit array are set to one, the element is reported as *possibly present*; if at least one bit is zero, the element is *definitely not present*. Due to hash collisions, Bloom Filters may produce false positives, but never false negatives.

In the sequential implementation, both insertion and querying iterate linearly over the input elements. The overall computational cost is dominated by the repeated evaluation of hash functions, resulting in a time complexity of $O(n \cdot k)$, where n is the number of processed elements. Memory usage is fixed and depends only on the chosen values of m and k.

2.2. Identification of Parallelizable Sections

The Bloom Filter algorithm exhibits a high degree of inherent parallelism. Each element is processed independently, both during insertion and querying, as hash computations for one element do not depend on the results of others. This makes the algorithm *embarrassingly parallel* at the level of input elements.

However, direct parallel updates to a shared bit array introduce potential data races and synchronization overhead, as multiple workers may attempt to set the same bit concurrently. While such races are logically harmless (setting a bit to one is an idempotent operation), their management in Python would require locks or atomic operations, significantly reducing performance.

To avoid fine-grained synchronization, the parallelizable sections are identified as the hash computation and index generation for each element, while the update of the global bit array is deferred. In a parallel design, each worker can process a subset of the input elements and construct a *local Bloom Filter* by setting bits in a private bit array. These partial results can then be merged safely in a reduction step using bitwise OR operations.

Similarly, during the query phase, membership tests for different elements can be performed independently on separate workers, as they only require read-only access to the Bloom Filter. This separation of independent computation and controlled aggregation forms the basis for the parallel implementation described in the following section.

3. Parallel Implementation

3.1. Parallelization Strategy

The parallel implementation adopts a hybrid strategy that combines `asyncio` and `multiprocessing` to effectively handle both I/O-bound and CPU-bound components of the Bloom Filter workload. The main idea is to separate file reading from hash computation and bit array manipulation, allowing these phases to overlap and scale independently.

Asynchronous programming through `asyncio` is used to manage file reading without blocking the main execution flow. Computationally intensive operations, such as hash evaluation and bit array updates, are instead parallelized using multiple processes through a `ProcessPoolExecutor`, thus bypassing the limitations imposed by Python's Global Interpreter Lock (GIL). The overall approach follows a *map–reduce* pattern, where independent portions of the input are processed in parallel and later aggregated.

3.2. Use of Multiprocessing in the Algorithm

Multiprocessing is employed in the parts of the algorithm where computational load is dominant, particularly during the Bloom Filter construction phase and the membership query phase. In both cases, input elements are divided into chunks, each assigned to a separate worker process.

During the construction phase, each worker processes its assigned subset of elements by computing the corresponding hash values and setting the associated bits in a local bit array. This design avoids concurrent access to a shared data structure, eliminating the need for locks or fine-grained synchronization. Once parallel process-

ing is completed, the local bit arrays produced by the workers are returned to the main process in serialized form and merged into a global Bloom Filter using bitwise OR operations, which are efficient and deterministic.

In the query phase, the global Bloom Filter is accessed in read-only mode. Membership tests for different elements can therefore be executed in parallel by separate workers without introducing synchronization issues. Chunking is again used to reduce scheduling overhead and to improve load balancing across processes.

The integration with `asyncio` enables efficient coordination of these parallel activities. The event loop manages asynchronous I/O operations and the submission of CPU-bound tasks to external executors, ensuring that the main execution flow remains responsive. Synchronization between processes is handled implicitly by waiting for all tasks to complete before proceeding to aggregation steps, keeping the implementation simple and free of race conditions.

This approach allows effective exploitation of process-level parallelism while limiting communication and synchronization overhead, and forms the basis for the performance improvements observed in the experimental evaluation.

4. Experimental Setup

4.1. Hardware

All experiments were conducted on a single workstation running a 64-bit operating system. The system is equipped with an **11th Gen Intel(R) Core(TM) i7-11800H** processor with a base frequency of **2.30 GHz**, featuring **8 physical cores** and **16 hardware threads**. The machine is equipped with **16 GB of RAM**, which is sufficient to store both the Bloom Filter bit arrays and the large input datasets entirely in main memory, avoiding disk swapping.

4.2. Software Environment

The entire project was developed in **Python**. Parallelism was implemented exclusively using standard Python libraries:

- `asyncio`, used for asynchronous I/O management and task coordination;
- `multiprocessing`, through `ProcessPoolExecutor`, used for parallel execution of CPU-bound tasks.

For hash computation within the Bloom Filter, the external library **MurmurHash3**, accessed via the Python package `mmh3`, was used. This library was chosen for its high performance and good hash value distribution. The Bloom Filter bit array is implemented using a dedicated bit array library that provides compact memory representation and efficient bitwise operations.

All experiments were executed using the same Python interpreter and library versions. Garbage collection was explicitly triggered between runs to reduce interference caused by automatic memory management.

4.3. Datasets

All datasets used in the experiments are **fully synthetic** and are generated using the script `generate_data.py`, located in the root directory of the project. The script is designed to produce controlled and reproducible workloads, allowing systematic variation of dataset size and input fragmentation.

For each configuration, two distinct datasets are generated:

- a **training dataset**, used to build the Bloom Filter;
- a **test dataset**, used during the membership query phase.

Each dataset element is a randomly generated alphanumeric string with a length uniformly distributed between 10 and 20 characters. Characters are sampled from a fixed alphabet consisting of lowercase letters and digits. Training and test datasets have the **same size** but contain **independent data**, with no intentional overlap between the two sets. As a result, most membership queries are expected to be negative, allowing the

evaluation of Bloom Filter behavior in the presence of false positives.

Generated datasets are stored in the directory `bench_data/`, which is organized hierarchically according to dataset size and fragmentation level. The directory structure is as follows:

```
bench_data/
  size_<N>/
      split_<S>/
          train/
              train_00.txt
              train_01.txt
              ...
          test/
               test_00.txt
               test_01.txt
               ...
```

Here, `size_<N>` denotes the total dataset size (2M, 5M, 10M, or 20M elements), while `split_<S>` represents the number of files used to store the dataset (1, 2, 4, 8, or 16). Dataset fragmentation does not alter the dataset content, but only its on-disk organization, enabling the study of I/O behavior and asynchronous file access.

4.4. Tested Parameters

The experimental evaluation systematically explores the following parameters:

- **Dataset size**: 2M, 5M, 10M, and 20M elements;
- **Number of worker processes**: 1, 4, 8, and 16;
- **Chunk size**: fixed values (20,000 and 50,000 elements) and an automatic mode;
- **Dataset fragmentation**: number of input files (splits).

Bloom Filter parameters, specifically the bit array size and the number of hash functions, are computed automatically as a function of dataset size and a target false positive rate of 1%.

4.5. Measurement Methodology

Performance measurements are carried out using the script `main.py`, which implements a complete and reproducible benchmark. For each configuration, both the **sequential implementation** (`sequential_bloom_filter.py`) and the **hybrid parallel implementation** (`hybrid_bloom_filter.py`) are executed using identical datasets and parameters.

Execution time is measured using the high-resolution wall-clock timer `time.perf_counter`. The reported execution time includes both the **Bloom Filter construction phase** and the **membership query phase**.

For each configuration, an initial **warm-up run** is performed and excluded from the measurements, followed by **three measured runs**. Reported values correspond to the average execution time across the measured runs. Speedup is computed as the ratio between the total execution time of the sequential implementation and that of the hybrid parallel implementation.

Final results are stored in CSV files (e.g., `results_final.csv` and `results_final_1.csv`) and are used for the performance analysis presented in the following section.

5. Results and Analysis

5.1. Correctness Validation

Correctness of the parallel implementation was verified by comparing its behavior against the sequential Bloom Filter baseline. For each experimental configuration, the hybrid implementation was executed using the same training and test datasets as the sequential version.

The validation focused on ensuring that:

- the Bloom Filter construction completed without errors;
- membership queries produced consistent results with the sequential implementation;

- no false negatives were observed, as guaranteed by the Bloom Filter properties.

Since training and test datasets were generated independently, most membership queries are expected to return negative results. Any positive response from the Bloom Filter was therefore interpreted as a false positive, which is acceptable by design. Identical query behavior between sequential and hybrid implementations confirmed the correctness of the parallelization strategy.

5.2. Performance Results

Performance results are reported in terms of execution time and speedup relative to the sequential baseline. Speedup is defined as:

$$\text{Speedup} = \frac{T_{\text{sequential}}}{T_{\text{parallel}}}$$

where $T_{\text{sequential}}$ and T_{parallel} denote the total execution time (build + query) of the sequential and hybrid implementations, respectively.

Across all dataset sizes, the hybrid implementation achieves substantial performance improvements when appropriate parameters are selected. Speedup increases with the number of worker processes up to a saturation point and improves significantly as dataset size grows.

5.3. Speedup Curves

Speedup curves were obtained by varying the number of worker processes while keeping the dataset size fixed. Figure 1 reports the speedup achieved by the hybrid implementation for different input sizes, using automatic chunk sizing and a single input file in order to isolate the effect of parallelism.

For all dataset sizes, speedup increases as the number of workers grows, up to a clear saturation point. For the smallest dataset (2M elements), speedup remains moderate, reaching a maximum of approximately $2.5\times$, indicating that multiprocessing overhead still represents a significant fraction of the total execution time.

As the dataset size increases, the effectiveness of parallelization improves substantially. For the 10M and 20M datasets, the hybrid implementation consistently achieves a stable speedup close to $4\times$ using 8 worker processes. This behavior shows that, for sufficiently large workloads, the computational cost dominates the parallelization overhead, allowing the available resources to be exploited efficiently.

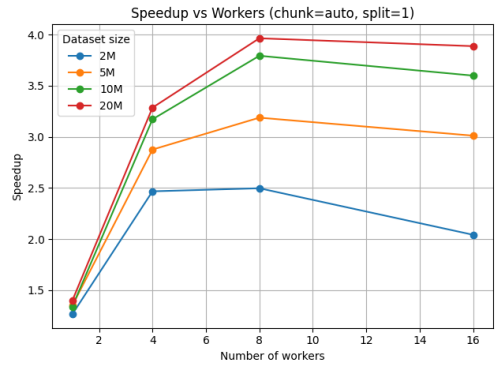

Figure 1. Speedup as a function of the number of worker processes for different dataset sizes using automatic chunking and a single input file.

Increasing the number of workers beyond 8 does not lead to further performance improvements and often results in a slight degradation. This effect is attributed to increased context switching and process management overhead, which offset the benefits of additional parallelism.

5.4. Scalability Analysis

The results provide clear evidence of effective **strong scaling**. When the problem size is doubled from 10M to 20M elements, the maximum achievable speedup remains nearly unchanged. This indicates that the parallel implementation scales well with increasing workload and that the overhead introduced by multiprocessing does not grow disproportionately with input size.

Scalability saturates around 8 worker processes, despite the availability of additional hardware threads. Beyond this point, the system becomes limited by operating system scheduling, memory contention, and process management costs, leading to diminishing returns and, in some cases, performance regression.

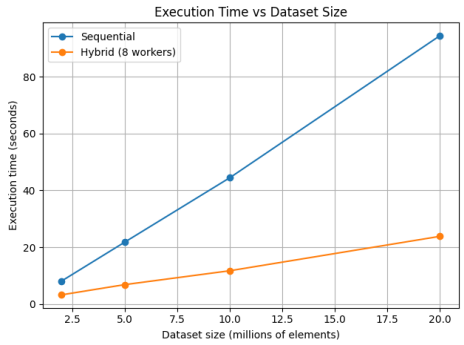

Figure 2. Execution time as a function of dataset size for the sequential and hybrid implementations. Results refer to a configuration with 8 worker processes and automatic chunk sizing for the hybrid version.

Overall, the scalability analysis confirms that the proposed hybrid approach is well balanced and particularly effective for large, CPU-bound workloads.

5.5. Effect of Different Parameters

This section analyzes the impact of different parameters on the performance of the hybrid Bloom Filter implementation. In particular, we focus on how execution time scales with the size of the input dataset and on the effect of chunk size selection. The influence of worker count and process management has already been discussed through the speedup curves presented in the previous subsection and is therefore not repeated here.

Execution Time Scaling with Input Size. To evaluate how execution time scales with increasing workload, we compare the total execution time of the sequential and hybrid implementations for representative configurations. Figure 2 reports the execution time as a function of the dataset size for the sequential baseline and for the hybrid implementation using 8 worker processes and automatic chunk sizing.

The results show that execution time grows approximately linearly with the size of the input dataset for both implementations. However, the hybrid approach exhibits a significantly lower growth rate, leading to an increasing performance gap as the dataset size increases. For small datasets, the overhead introduced by multiprocessing and task coordination partially offsets the benefits of parallelism. In contrast, for larger datasets (10M and 20M elements), the computational workload dominates the overhead, and the hybrid implementation achieves substantial reductions in execution time.

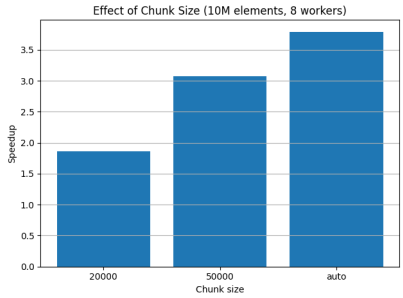

Figure 3. Effect of chunk size on speedup for a dataset of 10 million elements, using 8 worker processes and a single input split.

These results confirm that the proposed hybrid approach is particularly effective for large-scale, CPU-bound workloads, where the cost of parallelization is amortized over a sufficiently large amount of computation.

Effect of Chunk Size. Chunk size plays a crucial role in determining the overall efficiency of the parallel execution. Figure 3 illustrates the impact of different chunk size configurations on performance, measured in terms of speedup, for a fixed workload of 10 million elements using 8 worker processes and a single input split.

The results clearly show that small fixed chunk sizes (e.g., 20,000 elements) lead to poor performance due to excessive task scheduling and interprocess communication overhead. Larger fixed chunk sizes mitigate this effect but still require manual tuning. The automatic chunk sizing strategy consistently achieves the best or near-best performance by dynamically adapting the chunk size to the workload and available resources.

This behavior highlights the importance of balancing computational granularity and scheduling overhead. Automatic chunking provides a robust and adaptive solution that simplifies configuration while delivering stable and repeatable per-

formance improvements across different problem sizes.

5.6. Comparison of Parallelization Strategies and Bottleneck Analysis

This section compares the execution strategies adopted in this work and discusses the main factors that limit scalability. Since the project focuses on a single parallel design, the comparison is performed between the sequential baseline and the proposed hybrid parallel implementation.

Sequential vs Hybrid Parallel Strategy. The sequential implementation serves as a baseline reference and executes all phases of the Bloom Filter workflow—data loading, hash computation, filter construction, and membership queries—within a single process. This approach benefits from minimal overhead and simple control flow, but it does not scale to large datasets due to the CPU-bound nature of repeated hash computations.

The hybrid implementation combines asynchronous I/O and multiprocessing to address these limitations. Asynchronous file reading allows overlapping input operations with computation, while CPU-intensive tasks are distributed across multiple worker processes. This design bypasses the limitations imposed by Python's Global Interpreter Lock (GIL) and enables true parallel execution on multi-core systems.

Experimental results show that the hybrid approach significantly outperforms the sequential baseline for sufficiently large datasets. While the sequential version remains competitive for small inputs, the hybrid strategy achieves speedups close to $4\times$ for datasets of 10M and 20M elements when appropriate parameters are selected. This confirms that the adopted parallelization strategy is effective in reducing execution time for large-scale, CPU-bound workloads.

Bottleneck Analysis and Scalability Limits. Despite the observed performance improvements, the hybrid implementation does not scale indefinitely. The experimental results reveal a clear saturation point around 8 worker processes. Increasing the number of workers beyond this value does not lead to further speedup and often results in performance degradation.

This behavior is primarily due to the overhead associated with multiprocessing. As the number of worker processes increases, costs related to process management, context switching, memory contention, and inter-process communication grow significantly. These overheads eventually dominate the computational gains obtained from additional parallelism.

Another limiting factor is task granularity. When chunks are too small, the overhead of task scheduling and data transfer outweighs the computational work performed by each worker, leading to poor performance. This effect is particularly evident in configurations using small fixed chunk sizes, which in some cases result in execution times worse than the sequential baseline.

Finally, the operating system and runtime environment play a crucial role. On the tested system, optimal performance is achieved when the number of worker processes is approximately half of the available hardware threads. Beyond this threshold, system-level scheduling and resource contention limit scalability.

Overall, the bottleneck analysis indicates that the scalability of the proposed solution is constrained not by the Bloom Filter algorithm itself, but by the overhead of multiprocessing and system-level resource management. These results highlight the importance of balanced parallelism and careful parameter selection.

6. Discussion

6.1. Interpretation of Results and Theoretical Comparison

The experimental results demonstrate that the proposed hybrid parallelization strategy provides substantial performance improvements over the sequential baseline, particularly for large, CPU-bound workloads. Speedup increases with both the size of the input dataset and the number of worker processes, up to a clear saturation point.

From a theoretical perspective, the observed behavior is consistent with Amdahl's Law. While the Bloom Filter algorithm itself is highly parallelizable, certain portions of the execution—such as task coordination, inter-process communication, and result aggregation—remain inherently sequential. These components limit the maximum achievable speedup and explain the observed saturation around 8 worker processes.

At the same time, the results align well with Gustafson's Law: as the problem size increases from 10M to 20M elements, the achieved speedup remains stable at approximately $4\times$. This indicates that increasing the workload allows the parallel portion of the computation to dominate, effectively amortizing fixed overheads and improving scalability.

Overall, the hybrid approach exhibits effective strong scaling within the tested range, confirming that the chosen parallelization strategy is well suited for large-scale Bloom Filter operations.

6.2. Overhead Analysis, Limitations, and Lessons Learned

Despite the achieved performance gains, the results clearly highlight the impact of parallelization overhead. Multiprocessing introduces non-negligible costs related to process creation, task scheduling, context switching, and memory contention. These overheads are particularly visible for small datasets and for configurations using overly fine-grained task decomposition, where the parallel version may offer limited or no benefit compared to the sequential baseline.

Another important limitation arises from system-level constraints. On the tested platform, performance saturates when the number of worker processes exceeds approximately half of the available hardware threads. Beyond this point, additional parallelism increases contention rather than computational throughput.

A key lesson learned from this study is the importance of balanced parallelism. Optimal performance is achieved not by maximizing the number of worker processes, but by carefully tuning the interaction between workload size, task granularity, and available hardware resources. In this context, automatic chunk sizing proved to be a robust solution, consistently delivering near-optimal performance without manual tuning.

7. Conclusions

This work presented and evaluated a hybrid Bloom Filter implementation based on asynchronous I/O and multiprocessing in Python. The proposed approach successfully overcomes the limitations of the Global Interpreter Lock for CPU-bound tasks and enables effective parallel execution on multi-core systems.

The experimental evaluation shows that the hybrid implementation achieves stable and repeatable speedups close to $4\times$ for large datasets (10M and 20M elements), significantly reducing execution time compared to the sequential baseline. The best overall configuration consistently uses 8 worker processes combined with automatic chunk sizing, which balances computational efficiency and overhead.

These results confirm that the proposed hybrid strategy is an effective and scalable solution for large-scale Bloom Filter construction and querying in Python.

8. Future Work

Several extensions of this work are possible. Future developments could explore alternative data partitioning strategies, different process-to-core affinity configurations, or the use of shared-memory data structures to reduce inter-process communication overhead. Additionally, extending the implementation to distributed environments or integrating GPU acceleration for hash computation represents a promising direction for further improving scalability and performance.

References

[1] B. H. Bloom, *Space/Time Trade-offs in Hash Coding with Allowable Errors*, Communications of the ACM, vol. 13, no. 7, pp. 422–426, 1970.

[2] A. Z. Broder and M. Mitzenmacher, *Network Applications of Bloom Filters: A Survey*, Internet Mathematics, vol. 1, no. 4, pp. 485–509, 2004.

[3] G. M. Amdahl, *Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities*, AFIPS Conference Proceedings, vol. 30, pp. 483–485, 1967.

[4] J. L. Gustafson, *Reevaluating Amdahl's Law*, Communications of the ACM, vol. 31, no. 5, pp. 532–533, 1988.

[5] Python Software Foundation, *multiprocessing — Process-based parallelism*, Python 3 Documentation. Available: `https://docs.python.org/3/library/multiprocessing.html`